

Lab Assignment 10.1 – Code Review and Code Quality Improvement

Course: AI Assisted Coding

2303A51399

Assignment Type: Lab

Academic Year: 2025–2026

Question 1: Syntax and Logic Error Identification

Problem Statement

Detect and correct syntax and logical errors in the given Python program used to compute the average of student marks.

Input

Program to calculate average marks

```
def average_marks(marks_list):  
    total_sum = 0  
    for value in marks_list:  
        total_sum += value  
    avg = total_sum / len(marks_list)  
    return avrg
```

```
marks_list = [85, 90, 78, 92]  
print("Average =", average_marks(marks_list))
```

Answer (Corrected Code)

```
def average_marks(marks_list):  
    total_sum = 0  
    for value in marks_list:  
        total_sum += value  
    avg = total_sum / len(marks_list)  
    return avg
```

```
marks_list = [85, 90, 78, 92]
print("Average =", average_marks(marks_list))
```

Output

Average = 86.25

Code Explanation

- Corrected the misspelled variable avrg to avg.
 - Added the missing closing parenthesis in the print() function.
 - Ensured the function returns the computed average value correctly.
-

Question 2: PEP 8 Code Formatting

Problem Statement

Modify the given code to comply with PEP 8 coding standards.

Input

```
def rectArea(l,b):return l*b
print(rectArea(10,20))
```

Answer (PEP 8 Compliant Code)

```
def rectangle_area(length, width):
    return length * width
```

```
result = rectangle_area(10, 20)
print(result)
```

Output

200

Code Explanation

- Renamed function and variables to use lowercase with underscores.
- Added proper spacing around operators.
- Stored the function result in a variable for better readability.

Question 3: Improving Code Readability

Problem Statement

Rewrite the code to improve readability without changing its functionality.

Input

```
def f(a,b):  
    return a*b/100
```

```
x=200
```

```
y=15
```

```
print(f(x,y))
```

Answer (Readable Code)

```
def calculate_discount(price, discount_rate):  
    discount = (price * discount_rate) / 100  
    return discount
```

```
price = 200
```

```
discount_rate = 15
```

```
print(calculate_discount(price, discount_rate))
```

Output

```
30.0
```

Code Explanation

- Function name changed to describe its purpose clearly.
- Variables renamed for better understanding.
- Code layout improved with consistent indentation.

Question 4: Refactoring for Reusability

Problem Statement

Convert repetitive code into a reusable function.

Input

```
names = ["Alice", "Bob", "Charlie"]
```

```
print("Welcome", names[0])
```

```
print("Welcome", names[1])
```

```
print("Welcome", names[2])
```

Answer (Refactored Code)

```
def greet_students(names):
```

```
    for name in names:
```

```
        print("Welcome", name)
```

```
names = ["Alice", "Bob", "Charlie"]
```

```
greet_students(names)
```

Output

```
Welcome Alice
```

```
Welcome Bob
```

```
Welcome Charlie
```

Code Explanation

- Introduced a reusable function `greet_students`.
 - Removed hardcoded index-based access.
 - Improved scalability for larger student lists.
-

Question 5: Performance Enhancement

Problem Statement

Optimize the program to improve execution efficiency.

Input

```
numbers = [i for i in range(1, 1000000)]
```

```
result = []
```

```
for num in numbers:
```

```
    result.append(num * num)
```

```
print(len(result))
```

Answer (Optimized Code)

```
numbers = range(1, 1000000)
```

```
squared_numbers = [num * num for num in numbers]
```

```
print(len(squared_numbers))
```

Output

```
999999
```

Code Explanation

- Eliminated unnecessary list creation.
- Used list comprehension for faster execution.
- Reduced memory overhead and improved performance.

Question 6: Reducing Logical Complexity

Problem Statement

Simplify the nested conditional logic used to assign grades.

Input

```
def calculate_grade(marks):
```

```
    if marks >= 90:
```

```
        return "A"
```

```
    else:
```

```
        if marks >= 80:
```

```
            return "B"
```

```
        else:
```

```
            if marks >= 70:
```

```
                return "C"
```

```
            else:
```

```
                if marks >= 60:
```

```
                    return "D"
```

```
            else:
```

```
                return "F"
```

Answer (Simplified Code)

```
def calculate_grade(marks):
```

```
    if marks >= 90:
```

```
        return "A"
```

```
    elif marks >= 80:
```

```
        return "B"
```

```
    elif marks >= 70:
```

```
        return "C"
```

```
    elif marks >= 60:
```

```
        return "D"
```

```
return "F"
```

Output

Grade is returned based on marks input